

ANL Protocol — weryfikacja zmian po audycie #3

Zakres i ograniczenia

Zweryfikowałem bezpośrednio źródła w paczce `anl-protocol (2).zip`, zamiast ufać `CHANGES-AFTER-AUDIT3.md` i `TEST-LOG.txt`.

Paczka nie zawiera katalogu `.git`. W obecnym środowisku nie są dostępne `rustc`, `cargo`, `anchor`, `solana`, `cargo-audit` ani `cargo-deny`. Nie mogłem więc niezależnie odtworzyć kompilacji i testów.

Wyniki w `docs/TEST-LOG.txt` są dowodem pomocniczym, ale nadal nie są dowodem wykonanym przeze mnie ani kryptograficznie powiązanym z audytowaną paczką.

1. Status pięciu punktów checklisty

Punkt 1 — diffy i poprawki kodowe

Status: SPEŁNIONE statycznie

M-01: `ACCOUNT_VERSION` w `FundXnt`

Obie pule mają teraz właściwy constraint:

```
constraint = genesis_pool.version == ACCOUNT_VERSION
@ AnlError::InvalidAccountVersion
```

oraz odpowiednik dla Flexible.

Dowód:

- `programs/anl_staking/src/instructions/fund.rs:124-140`

M-01 jest poprawione w źródle.

L-01: owner check checkpointów

Jawne sprawdzenie właściciela zostało dodane w obu miejscach:

- `settlement_cap_index` :
• `programs/anl_staking/src/instructions/lifecycle.rs:69-72`
- `roll_checkpoint` :
• `programs/anl_staking/src/instructions/fund.rs:196-208`

Kod wymaga:

```
require_keys_eq!(*info.owner, *program_id, AnlError::CheckpointMismatch);
```

L-01 jest poprawione.

H-01: blokady feature'ów

Dodano:

```
#[cfg(all(feature = "network-mainnet", feature = "test-periods"))]  
compile_error!("test-periods cannot be enabled together with network-  
mainnet");  
  
#[cfg(all(feature = "network-mainnet", feature = "network-testnet"))]  
compile_error!("select exactly one network feature");
```

Dowód:

- `programs/anl_staking/src/lib.rs:11-15`
- `programs/anl_staking/Cargo.toml:11-18`

Blokady są poprawnie zadeklarowane w kodzie.

Ocena punktu 1

Spełnione statycznie. Nie mogłem potwierdzić kompilacją, ale hunki znajdują się w rzeczywistych źródłach i wyglądają poprawnie.

Punkt 2 — surowe logi, czyste drzewo i powiązanie z HEAD

Status: CZĘŚCIOWE / mechanizm dowodowy wymaga poprawy

`docs/TEST-LOG.txt` zawiera deklarowane wyniki:

- `anl-math` : 24/24 w obu wariantach;
- `core` : 34/34;
- integracja: 4/4 w obu wariantach;
- Clippy: zakończony;
- negatywne buildy: zakończone oczekiwanym błędem.

Dowód:

- `docs/TEST-LOG.txt` : 22-60

Jednak obecny log ma kilka problemów.

2.1 `cargo fmt --check` w pokazanym logu nie jest czysty

Log rozpoczyna się od rzeczywistego diffu `cargo fmt`:

- `docs/TEST-LOG.txt:7-20`

To oznacza, że przy tym konkretnym wykonaniu:

```
cargo fmt --all --check
```

znalazł niezformatowany plik.

Log nie pokazuje kodu wyjścia tej komendy, więc deklaracji „fmt czysty” nie można wyprowadzić z tego pliku.

Kod w przesłanej paczce może już być sformatowany, ale sam załączony log tego nie dowodzi.

2.2 `audit-evidence.sh` nie przerywa po nieudanym poleceniu

Skrypt ma:

```
set -uo pipefail
```

zamiast:

```
set -euo pipefail
```

Dowód:

- `scripts/audit-evidence.sh:4`

Funkcja:

```
run(){  
    echo "$ $*" >>"$LOG"  
    "$@" >>"$LOG" 2>&1  
    echo "exit: $?" >>"$LOG"  
}
```

jedynie zapisuje kod wyjścia i kontynuuje.

Dowód:

- `scripts/audit-evidence.sh:5-6`

W rezultacie:

- `fmt` może nie przejść;
- `Clippy` może nie przejść;
- testy mogą nie przejść;
- `cargo audit` może nie przejść;
- `cargo deny` może nie przejść;

a skrypt nadal zakończy się komunikatem:

```
GOTOWE
```

i prawdopodobnie kodem `0`.

To nie realizuje wymagania „pełny bieg dowodowy musi przejść”.

2.3 Kontrola czystego drzewa nie jest egzekwowana

Skrypt wykonuje:

```
git status --porcelain
```

ale nie sprawdza, czy output jest pusty.

Dowód:

- `scripts/audit-evidence.sh:16-19`

Co więcej, przed sprawdzeniem statusu skrypt nadpisuje śledzony plik:

```
docs/TEST-LOG.txt
```

Dowód:

- `scripts/audit-evidence.sh:5`
- `scripts/audit-evidence.sh:8-14`

Jeżeli `TEST-LOG.txt` jest śledzony przez Git, samo rozpoczęcie audytu powoduje zmianę drzewa. `git status --porcelain` prawdopodobnie wykaże zmodyfikowany plik, ale skrypt to zignoruje.

2.4 Sam zapis `HEAD` nie jest powiązaniem kryptograficznym logu

Skrypt zapisuje:

```
git rev-parse HEAD
```

To identyfikuje commit, ale nie dowodzi, że:

- drzewo było czyste;
- wszystkie polecenia przeszły;
- log nie został później ręcznie zmieniony;
- załączona paczka odpowiada temu commitowi.

Lepszy model:

1. sprawdzić czyste drzewo przed utworzeniem logu;
2. pisać log do pliku tymczasowego poza repo;
3. przerwać przy pierwszym nieoczekiwanym błędzie;
4. po zakończeniu zapisać:
5. HEAD;
6. `git status --porcelain`;
7. hash logu;
8. hash źródłowego archiwum;
9. hash binarki;
10. podpisać manifest lub przynajmniej commit/tag release.

Zalecany rdzeń skryptu

```
set -euo pipefail

test -z "$(git status --porcelain)" || {
    echo "Working tree is not clean"
    exit 1
}

HEAD="$(git rev-parse HEAD)"
LOG="$(mktemp)"

run() {
    echo "\$ $*" | tee -a "$LOG"
    "$@" 2>&1 | tee -a "$LOG"
}
```

Dla testów negatywnych należy osobno sprawdzić zarówno kod niezerowy, jak i właściwy komunikat.

Ocena punktu 2

Częściowe. Log zawiera wartościowe dane, lecz obecny skrypt nie gwarantuje sukcesu testów ani czystości źródła.

Punkt 3 — negatywne dowody i job `release-guards`

Status: W DUŻEJ MIERZE SPEŁNIONE

Mainnet + `test-periods`

Job:

1. zapisuje output;
2. sprawdza niezerowy kod wyjścia;
3. sprawdza konkretny komunikat `compile_error!`;
4. failuje, gdy build niespodziewanie przechodzi albo pada z innego powodu.

Dowód:

- `.github/workflows/ci.yml:46-59`

To jest poprawnie skonstruowany test negatywny.

Mainnet + testnet jednocześnie

Job sprawdza, czy komenda nie przechodzi:

- `.github/workflows/ci.yml:60-65`

Jest to lepsze niż brak testu, ale słabsze niż pierwszy guard, ponieważ nie sprawdza konkretnego powodu błędu. Build mógłby kiedyś paść z niezwiązanej przyczyny, a test zostałby uznany za zaliczony.

Zalecana poprawka

Drugi test powinien również przechwycić output i wymagać:

```
select exactly one network feature
```

tak jak pierwszy wymaga komunikatu H-01.

Pozytywne warianty

CI kompiluje również:

```
cargo check -p an1_staking --features network-mainnet
cargo check -p an1_staking --features "network-testnet,test-periods"
```

Dowód:

- `.github/workflows/ci.yml:66-69`

To zapobiega sytuacji, w której wszystkie warianty są blokowane przez błędne cfg.

Ocena punktu 3

Spełnione z drobnym hardeningiem. H-01 ma poprawny guard. Drugi negatywny test powinien sprawdzać właściwy komunikat.

Punkt 4 — polityka feature'ów i release

Status: CZĘŚCIOWE; politykę można obejść proceduralnie

Decyzja o pozostawieniu domyślnego builda bez feature'u sieci nie tworzy sama w sobie bezpośredniej podatności on-chain.

Logiczna implikacja:

```
network-mainnet ⇒ brak test-periods
```

jest poprawnie egzekwowana przez `compile_error!`.

Ponadto kontrola `EXPECTED_XNT_MINT` jest aktywna dla każdego builda bez `test-periods`:

- `programs/an1_staking/src/instructions/initialize.rs:85-92`

Zatem prawidłowy `network-mainnet` nie może wyłączyć kontroli minta przez dodanie `test-periods`.

Problem: feature'y sieci nie sterują obecnie logiką sieciową

`network-mainnet` i `network-testnet` są używane wyłącznie w dwóch `compile_error!`.

Nie wpływają bezpośrednio na:

- Program ID;
- oczekiwany mint;
- cluster;
- stałe sieciowe;
- zachowanie runtime poza wzajemnym wykluczeniem.

W rezultacie zwykły build:

```
anchor build
```

bez jakiegokolwiek feature'u sieci nadal tworzy produkcyjny wariant czasowy z aktywną kontrolą `EXPECTED_XNT_MINT`.

Taki artefakt nie jest oznaczony jako `network-mainnet`, ale technicznie może zostać wdrożony.

Problem: README nadal zaleca obejście skryptów release

README pokazuje:

```
anchor build -- --features test-periods
```

bez `network-testnet`.

Dowód:

- `README.md:65-72`
- `README.pl.md:67-74`

Dalej pokazuje:

```
anchor build
```

jako zwykłą ścieżkę budowania.

Dowód:

- `README.md:74-81`
- `README.pl.md:76-83`

To jest bezpośrednio sprzeczne z deklaracją:

```
release wyłącznie przez scripts/build-mainnet.sh /  
scripts/build-testnet.sh.
```

Polityka nie jest wiarygodna, dopóki główna dokumentacja uczy alternatywnej ścieżki.

Problem: `build-mainnet.sh` nie sprawdza całego brudnego drzewa

Skrypt używa:

```
git diff --quiet
```

Dowód:

- `scripts/build-mainnet.sh:4-6`

To nie wykrywa niezacomitowanych zmian znajdujących się w indeksie:

```
git add modified_file
```

Nie wykrywa też plików untracked.

Poprawna kontrola:

```
test -z "$(git status --porcelain)"
```

Problem: `build-testnet.sh` nie sprawdza czystości w ogóle

- `scripts/build-testnet.sh:1-4`

Można wygenerować manifest wskazujący na `HEAD`, chociaż binarka powstała z lokalnie zmodyfikowanego kodu.

Problem: manifest nie dowodzi sposobu budowania

Manifest zapisuje tekst:

```
features: network-mainnet
```

ale jest to wartość zmiennej ustawionej w skrypcie. Nie jest ona pobierana z metadanych binarki.

To rozwiązanie jest użyteczne proceduralnie, ale pozostaje zależne od:

- zaufania do skryptu;
- czystości drzewa;
- braku późniejszej podmiany binarki;
- kontroli procesu deploymentu.

Czy brak trzeciego `compile_error!` jest akceptowalny?

Tak, warunkowo, ale tylko jeżeli:

1. główna dokumentacja usuwa wszystkie zwykłe instrukcje `anchor build`;
2. CI/release wymaga użycia jednego ze skryptów;
3. skrypty odrzucają każde brudne drzewo;
4. deployment pobiera wyłącznie artefakt z chronionego joba release;
5. hash wdrażanej binarki jest porównywany z manifestem;
6. ręczny deployment z lokalnego artefaktu jest zabroniony organizacyjnie.

W aktualnym stanie polityka pozostaje obchodliwa przez:

```
anchor build  
anchor deploy
```

lub przez zbudowanie z lokalnie zmodyfikowanego, ale staged drzewa.

Ocena punktu 4

Częściowe. Blokada `mainnet + test-periods` jest dobra, ale egzekwowanie ścieżki release nie jest jeszcze szczelne.

Punkt 5 — dokumentacja i repo-wide grep

Status: CZĘŚCIOWE

Semantyka `end_ts` kontra `end_epoch`

Oba README opisują teraz poprawną regułę:

- ANL kończy się dokładnie na `end_ts` ;
- XNT obejmuje pełne `end_epoch` .

Dowód:

- README.md: 19-20
- README.md: 87
- README.pl.md: 19-20
- README.pl.md: 89

Nie znalazłem wcześniejszego sformułowania, że oba strumienie kończą się na `end_ts` .

Stare liczby testów

Nie ma już ciągów `23/23` i `3/3` , ale w obu README nadal występuje nieaktualna liczba 23:

```
cargo test -p anl-math # math (23)
```

Dowód:

- README.md: 74-80
- README.pl.md: 76-82

Tymczasem tabela deklaruje 24/24:

- README.md: 31
- README.pl.md: 31

Zatem twierdzenie „repo-wide grep nie znajduje już starej liczby 23” jest zbyt szerokie. Nie znajduje dokładnego ciągu `23/23` , lecz nieaktualne `23` nadal pozostało.

Dalsza nieaktualna dokumentacja bezpieczeństwa

README nadal opisuje zabezpieczenia `test-periods` głównie jako:

- brak feature’u w default;
- ostrzegawczy log.

Dowód:

- README.md: 70-72

- `README.pl.md:72-74`

Nie opisuje nowego wymogu użycia:

- `network-mainnet` ;
- `network-testnet` ;
- skryptów release;
- compile-time guards.

Dokumentacja powinna zostać zaktualizowana, by użytkownik nie stosował starego procesu.

Ocena punktu 5

Częściowe. Mismatch `end_ts/end_epoch` poprawiono, lecz pozostały stare instrukcje builda i liczba 23.

2. Zmiany nazw handlerów i `epoch_of`

Nazwy handlerów

Status: brak widocznej regresji statycznej

Publiczne instrukcje programu nadal nazywają się:

- `initialize` ;
- `create_pool` ;
- `stake` ;
- `fund_rewards` ;
- `fund_xnt` ;
- `set_operator` ;
- `settle_expired` ;
- `claim` ;
- `unstake_early` ;
- `pause` ;
- `resume` .

Dowód:

- `programs/anl_staking/src/lib.rs:28-91`

Zmianie uległy jedynie wewnętrzne funkcje implementacyjne:

- `initialize_handler` ;
- `create_pool_handler` ;
- `stake_handler` .

Dowód:

- `programs/anl_staking/src/instructions/initialize.rs:82`

- `programs/anl_staking/src/instructions/create_pool.rs:31`
- `programs/anl_staking/src/instructions/stake.rs:86`

Anchor generuje powierzchnię instrukcji na podstawie funkcji znajdujących się w module `#[program]`, nie na podstawie nazw wewnętrznych helperów.

Ponieważ nazwy funkcji w `#[program]` pozostały bez zmian, nie ma statycznej podstawy do oczekiwania zmiany discriminatorów instrukcji.

Zmiana usuwa także kolizję z glob-reeksportami, zachowując typy `Context` i argumenty.

Nie mogłem jednak potwierdzić IDL ani porównać wygenerowanych discriminatorów, ponieważ `anchor` nie jest dostępny. Przed deploymentem warto porównać poprzedni i nowy plik IDL.

`epoch_of` → `Option<u64>`

Status: zmiana wygląda poprawnie

Definicja:

- `programs/anl_staking/src/state/mod.rs:238`

Callsite fundingu konwertuje `None` na `BeforeGenesis`:

- `programs/anl_staking/src/instructions/fund.rs:231-235`

Callsite pozycji także obsługuje `None` jako błąd:

- `programs/anl_staking/src/instructions/stake.rs:195-196`

Nie znaleziono ignorowanego `Option`, `unwrap()` ani cichego fallbacku. Zmiana poprawia model błędu i nie ujawnia regresji statycznej.

3. Dodatkowe ustalenia dotyczące CI

[M-EVIDENCE-01] `supply-chain` jest nieblokujący

Severity: MEDIUM procesowo

CI uruchamia:

```
cargo audit || true
```

oraz:

```
cargo deny check advisories bans sources || true
```

Dowód:

- `.github/workflows/ci.yml:71-83`

Oznacza to, że:

- wykryta podatność;
- zabroniona zależność;
- nieakceptowana licencja lub source policy;
- błąd samego narzędzia

nie powodują czerwonego CI.

Twierdzenie „cargo audit/deny wykonują się w jobie supply-chain” jest technicznie prawdziwe, ale job nie egzekwuje ich wyniku.

Patch

Usunąć `|| true`:

```
- name: cargo audit
  run: cargo audit

- name: cargo deny
  run: cargo deny check advisories bans sources
```

Dodać zatwierdzony `deny.toml` do repo zamiast generować go podczas CI.

[M-EVIDENCE-02] `audit-evidence.sh` może raportować GOTOWE po awariach

Severity: MEDIUM procesowo

Opisane wcześniej problemy:

- brak `set -e`;
- brak egzekwowania czystego drzewa;
- log tworzony przed kontrolą statusu;
- brak końcowego kodu niezerowego po błędzie.

Dopóki skrypt nie zostanie poprawiony, nie może być traktowany jako automatyczny certyfikat poprawnego biegu.

4. Zaktualizowany werdykt testnetowy

Warunkowo gotowy do zamkniętego testnetu

Zmiany po audycie poprawiają stan projektu:

- M-01 naprawione;
- L-01 naprawione;
- H-01 naprawione na poziomie cfg;
- warianty sieciowe są sprawdzane w CI;
- model checkpointów nie wykazuje nowej regresji statycznej;
- wewnętrzne zmiany handlerów nie zmieniają widocznej powierzchni instrukcji.

Przed testnetem należy jeszcze:

1. poprawić `audit-evidence.sh`;
2. poprawić stare instrukcje builda w README;
3. poprawić liczbę `23` na `24`;
4. usunąć `|| true` z supply-chain CI albo jawnie oznaczyć job jako informacyjny;
5. utworzyć testnetowy Program ID;
6. wykonać testy na realnym repo Git i zapisać HEAD;
7. zbudować przez poprawiony `build-testnet.sh`;
8. zweryfikować hash faktycznie wdrażanej binarki;
9. zachować upgrade authority;
10. używać aktywów bez wartości lub ściśle limitowanej ekspozycji.

Po tych zmianach zamknięty testnet jest rozsądnym następnym etapem.

5. Zaktualizowany werdykt immutable-mainnet

Nadal NIEGOTOWY

Nie zgadzam się jeszcze, że wszystkie pozostałe pozycje są „wyłącznie wdrożeniowe”.

Pozostają także realne problemy z pipeline’em dowodowym i polityką release:

- `audit-evidence.sh` nie failuje po błędach;
- skrypt nie egzekwuje czystego drzewa;
- `build-mainnet.sh` nie wykrywa staged ani untracked zmian;
- `build-testnet.sh` nie sprawdza czystości;
- README nadal pokazuje buildy omijające skrypty release;
- supply-chain CI ignoruje negatywne wyniki;
- załączony `TEST-LOG.txt` pokazuje diff z `cargo fmt --check`;
- paczka nadal nie ma identyfikowalnego commita;
- nie wykonano niezależnego verifiable build;
- Program ID pozostaje placeholderem;
- nie potwierdzono zgodności finalnego IDL i binarki.

Po naprawieniu powyższych problemów pozostałe blokery będą w większości wdrożeniowe:

1. finalny Program ID;
2. czysty, otagowany commit;
3. pełny bieg CI na tym commicie;
4. `cargo audit` i `cargo deny` jako blokujące;
5. verifiable/reproducible build;
6. hash binarki;
7. porównanie binarki z wdrożeniem;
8. finalny audyt artefaktu release;
9. kontrolowana polityka upgrade authority;
10. okres obserwacji na testnetcie.

6. Podsumowanie statusów

Punkt	Status
1. Poprawki M-01/L-01/H-01 w źródłach	Spełnione statycznie
2. Logi + czyste drzewo + HEAD	Częściowe; skrypt dowodowy wadliwy
3. Negatywne buildy i <code>release-guards</code>	Spełnione z drobnym hardeningiem
4. Polityka feature'ów/release	Częściowe; możliwe obejście proceduralne
5. Dokumentacja i repo-wide grep	Częściowe; semantyka poprawiona, stare buildy i liczba 23 pozostały
Handlery	Brak widocznej regresji statycznej
<code>epoch_of -> Option<u64></code>	Brak widocznej regresji statycznej
Zamknięty testnet	Warunkowo gotowy po poprawie pipeline'u
Immutable mainnet	Niegotowy

Ostateczny werdykt

Zmiany kodowe wskazane w audycie #3 zostały wdrożone poprawnie na poziomie źródła i nie ujawniły nowej regresji w checkpointach ani powierzchni instrukcji. Największym pozostałym problemem nie jest obecnie logika stakingu, lecz fakt, że skrypty i CI nie zapewniają jeszcze wiarygodnego, fail-closed łańcucha dowodowego od czystego commita do wdrożonej binarki.